MODERATED LEARNING MANAGEMENT SYSTEM

Final Software Engineering Report | CS4433

National University of Lesotho • Individual Project • Release v1.1

Student: Itumeleng Hatane | Student No.: 202322611

1. Project overview and aims

This project built an offline desktop Learning Management System (LMS) for one secondary school. Its main rule is that marks given by a lecturer are checked by the Head of Department (HoD) before students can see them. There are four roles: administrator, lecturer, HoD and student. The system covers users, departments, courses, enrolment, assignments, submissions and resubmissions, extensions, late penalties, marking, moderation, release of marks, audit records and backup.

The aim was not only a working program. The aim was a system that is easy to test and to change, with requirements that can be traced to code and tests, controlled versions and measured test results. The tools were C++17, Qt 6 Widgets, SQLite, CMake, libsodium, GoogleTest, Git and GitHub Actions.

2. Development process and planning

A phase-based Waterfall process was used, with the V-Model to link each phase to a test level. Eight phases were planned, one per week: planning, requirements, design, implementation, testing, quality assurance, deployment and maintenance. The Phase 1 plan had a work breakdown structure, a Gantt chart, milestones (gates) and a risk register. The risk register was reviewed at each gate. For example, risk R3 (unclear penalty and moderation rules) was handled in Phase 2 by writing exact business rules. Because one student did all the work, the moderated marking path was built first, and tests were written alongside the code.

Figure 1. Gantt chart, dependencies and milestones. View diagram ↗️

3. Requirements engineering

Stakeholders were placed on a power/interest grid. Four conflicts and eight unclear statements were found in the interviews and each was resolved. For example, marks stay hidden until the HoD releases them, and late work is accepted but penalised. The Software Requirements Specification (SRS) follows ISO/IEC/IEEE 29148. It has functional requirements FR-01 to FR-24, non-functional requirements NFR-01 to NFR-11 and business rules BR-01 to BR-08. Each has a MoSCoW priority and a test method. The non-functional requirements cover security, speed, offline use, usability and reliability.

Sixteen use cases (UC-01 to UC-16) describe what each actor does. Three are fully dressed: submit work (UC-10), mark work (UC-08) and moderate marks (UC-12). Class-wide extensions were first recorded as a “Won’t” item (FR-23). They were added later as change request CR1, which shows how requirements change after the baseline.

Figure 2. Actors and use cases. View diagram ↗️

4. Software architecture and design

A layered architecture was used. The top layer has two user interfaces: a console menu for each role (src/ui) and Qt windows (src/gui). The middle layer has the services that carry out the use cases and check permissions: AuthService, AdminService, AssignmentService, SubmissionService, MarkingService, ModerationService, AuditService and BackupService. The bottom layer (src/data) has the SQLite repositories, a Database class and a Transaction class. Domain classes (src/domain) hold the entities and their rules. The System class (src/app) creates

all services and repositories in one place. Both interfaces call the same services, so each rule is written only once.

Services depend on interfaces, not on SQL. For example, AssignmentService uses IAssignmentRepository and ICourseRepository. This lets tests use fake repositories and keeps database changes in one layer. The cost is more interfaces and more wiring. This was accepted because testing and maintenance are part of the project. MVC and client-server designs were considered and rejected for a program that runs on one computer.

Figure 3. Layered component architecture. View diagram ↗

4.1 Object-oriented design and patterns

Each service has one job (Single Responsibility Principle), and services depend on abstractions (Dependency Inversion Principle). Three design patterns are used:

- **Strategy:** IPenaltyPolicy with TenPercentPerDayPolicy, so the penalty rule can change without changing SubmissionService.
- **Repository:** I*Repository interfaces with Sqlite* classes that hold all SQL.
- **Factory:** UserFactory creates the right kind of user for each role.

IClock and IPasswordHasher (Argon2Hasher) are also interfaces, so time and hashing can be replaced in tests.

Figure 4. Domain classes. View diagram ↗

Figure 5. Services, interfaces and patterns. View diagram ↗

4.2 Behaviour models

The state machine shows a submission moving from Submitted or Late to Marked, Moderated and Released. Two activity diagrams show the student submission path and the marking, moderation and release path. Two sequence diagrams show “Submit assignment” and “Moderate and release marks”, including where permission and deadline checks happen.

Figure 6. Submission state machine. View diagram ↗

Figure 7a. Activity: submitting work. View diagram ↗ *Figure 7b. Activity: marking, moderation and release.* View diagram ↗

Figure 8. Sequence: student submits work. View diagram ↗

Figure 9. Sequence: HoD moderates and releases marks. View diagram ↗

5. Implementation, database and security

The code is split into src/domain, src/services, src/data, src/app, src/ui and src/gui (105 source and header files). Domain classes such as User, Assignment and Submission check their own values. Services check who is allowed to do each action. Errors are shown to the user as clear messages, and the program does not stop.

The database is one SQLite file. Its tables are department, user, course, course_lecturer, enrolment, assignment, extension, submission, submission_version and audit_entry, linked by foreign keys. Every version of a submission is kept. FileStore copies each submitted file into a submissions folder. If the database save fails, the copied file is deleted so files and records stay the same.

Figure 10. Entity relationship diagram. View diagram ↗

5.1 Business rules and access control

- Late work is accepted. The penalty is 10% of the maximum mark for each started 24 hours, for up to 3 days. Work more than 3 days late gets 0. A mark never goes below 0.
- An extension is for one student, is at most 14 days, needs a reason and recalculates lateness.

- Resubmission is allowed only before the effective deadline and before marking. The latest version is marked and all versions are kept.
- Marking needs written feedback. Marks are hidden from students until they are released.
- The HoD approves a mark or changes it with a reason, which is audited. Marks are released only when every submission is Moderated.

These rules are checked in the services, not only by hiding buttons. Passwords are stored as Argon2id hashes (libsodium). An account is locked for 5 minutes after 5 failed logins, and the same message is shown for a wrong username or password. The audit trail records who changed a mark, when, the old and new mark, and the reason. Changes with several steps run inside one database transaction.

6. Requirements-to-design traceability

- Login and roles (FR-01, FR-02) → AuthService, Argon2Hasher → AuthServiceTest, ST01, ST02
- Administration and enrolment (FR-03 to FR-05) → AdminService, SqliteCourseRepository → AdminServiceTest, ST03 to ST05
- Assignments and extensions (FR-06 to FR-08) → AssignmentService → AssignmentServiceTest, ST06, ST07
- Submission and late penalty (FR-09 to FR-13) → SubmissionService, TenPercentPerDayPolicy → SubmissionServiceTest, PenaltyPolicyTest, ST08, ST09
- Moderation and release (FR-14 to FR-17) → ModerationService → ModerationServiceTest, ST10
- CR1 class-wide extension (FR-23) → AssignmentService::extendClassDeadline, LecturerWindow → four new service tests

The full matrix, with source files and test names for every requirement, is in docs/RTM.md.

7. Verification, validation and quality assurance

After CR1, CTest ran **218 tests: 218 passed and 0 failed, in 157.18 seconds**. These are 204 GoogleTest tests and 14 system tests (ST01 to ST14). A separate GUI acceptance program ran 16 checks (GT-01 to GT-16) on the Qt windows, with 0 failures. The system tests gave the program 64 invalid inputs. All were refused with a message and the program never stopped (NFR-07 asks for 30). Ten defects were found during testing. All were fixed and closed (docs/DEFECT_LOG.md).

In Phase 5, gcovr measured **97.6% line coverage of src/services (580 of 594 lines)**. This is the Phase 5 figure. It was not measured again after CR1. In Phase 6, cppcheck 2.21.0 checked 44 source files and found 3 warnings. One was fixed; 2 style warnings were reviewed and kept, with 0 errors. Lizard 1.24.3 found no function above the complexity limit of 15. A self code review used a checklist and the project coding standard. These results show tested behaviour. They do not prove that the software has no defects.

7.1 Test levels

- **Unit tests:** domain rules, the penalty policy and password hashing.
- **Service tests:** each service with fake repositories, a fake clock and a fake audit store.
- **Integration tests:** services and repositories on a real SQLite database.
- **System tests:** the console program is run with typed input and its output is compared with the expected output.
- **Acceptance tests:** the GUI checks follow the main user tasks for all four roles.

After CR1 the whole suite was run again as a regression test.

8. Configuration management and CI

Git was used for version control and GitHub hosts the private repository. The main branch is the integration branch. Phase 6 and Phase 8 work was done on the branches feature/phase6-quality-assurance and feature/phase8-class-extensions, and merged through pull requests #1 and #2. The tags phase4-submitted, gui-8d, v1.0 and v1.1 mark the baselines. Tag v1.0 is the Phase 7 release. Tag v1.1 is the maintenance release with CR1 and the updated traceability matrix. The GitHub Actions workflow (.github/workflows/build.yml) runs on every push and pull request. It builds and tests on Ubuntu and on Windows, and a coverage job fails if line coverage drops below 70%. All six checks passed on the final revision of pull request #2.

9. Deployment and limits

The LMS runs offline on one Windows computer. No cloud service is used. The file Moderated_LMS_Windows.zip contains lms.exe (console), lms_gui.exe (Qt), the Qt libraries and plugins and other needed DLLs. It contains no database, submissions or backups. docs/deployment has the installation guide, the user manual for the four roles, the release notes and a deployment diagram. The ZIP was extracted into a clean folder on the development computer. The program started with no missing DLLs, created lms.db and the submissions folder, and showed the first-time administrator setup.

Limits: the package was not tested on a second Windows computer without the build tools. The SQLite file is for one computer, not for many computers at the same time. The school must keep backups in a safe place.

10. Software maintenance: change request CR1

CR1 asked that a lecturer can extend an assignment deadline for the whole class. It is perfective maintenance, because it adds new behaviour after release v1.0. The impact analysis found these parts: Assignment, AssignmentService, LecturerWindow and the lateness of existing submissions. The main risks were losing individual extensions and changing work that was already marked.

The new method AssignmentService::extendClassDeadline checks that the lecturer teaches the course, that the new deadline is later but not more than 14 days later, and that a reason is given. Inside one transaction it saves the new deadline and recalculates lateness for open (Submitted or Late) submissions. It uses each student’s effective deadline, so individual extensions stay. Marked, Moderated and Released work is not changed. A button was added to the lecturer window. Four tests were added: ClassWideExtensionUpdatesDeadline, ClassWideExtensionPreservesIndividualExtension, ClassWideExtensionRecalculatesLateSubmission and ClassWideExtensionPreservesMarkedSubmission. The work was done on feature/phase8-class-extensions and merged through pull request #2, then tagged as release v1.1. After the change, 218 of 218 tests passed, the GUI checks had 0 failures and all CI checks passed. One GUI check (GT-09) had to be corrected to use dates relative to the deadline.

11. Professional, legal and ethical issues

Marks and passwords are personal data, so the Lesotho Data Protection Act 2011 applies. Each role can see and do only what it needs. Only password hashes are stored. The audit trail makes every HoD change reviewable. The same penalty rule applies to every student, and late work still gets feedback, which keeps assessment fair. Demonstrations use made-up accounts, not real student data. Qt (LGPL), libsodium, SQLite and GoogleTest have licence terms that must be

followed when the program is shared. Test results are reported as measured, including their limits.

12. Critical evaluation and conclusion

The project followed the full life cycle: plan, SRS, UML design, layered C++ code, automated testing, version control, packaging and maintenance. Its strengths are clear layers, services that are easy to test, moderation with an audit trail, and tests that link back to requirements. Its limits are: one computer only; the class-wide extension reason is checked but not saved in the audit trail; coverage was not measured again after CR1; the package was not tested on another computer; and the “Could” items FR-21 and FR-22 were deferred. Future work could store the CR1 reason in the audit trail, test backup and restore more fully, and move to a server database if many computers must use the system at once.

References and repository evidence

CS4433 project brief; Phase 1 project plan; Phase 2 SRS; Phase 3 design; Phase 4 to Phase 8 records in the repository (docs/, evidence/phase8/). ISO/IEC/IEEE 29148:2018, Requirements engineering. Sommerville, I. (2016) Software Engineering, 10th edn. Pearson.

Source repository: GitHub LMS repository

Release: v1.1 release tag

Diagram images and their editable PlantUML sources are stored in docs/diagrams/ in the repository.